TERRALENS — The Lens Built for Change

Tagline: Discover Earth's Changes. Understand Their Impact.

A gamified Earth observation platform powered by NASA–ISRO's NISAR satellite that allows players to investigate real changes happening on Earth's surface using radar imagery.

Players become Earth Investigators. They receive missions, explore satellite observations from different dates, detect environmental changes, and understand how those changes affect nearby communities.

Instead of focusing mainly on building a village, TERRALENS makes radar investigation the heart of the game. Players earn rewards by discovering and explaining real changes on Earth.

The 4 Core Features

Assam flood: Isro's Nisar satellite captures Brahmaputra's might from space - India Today

1. Earth Explorer — Explore the Radar

Choose real locations around the world and explore available NISAR radar images. View different dates and learn how radar observes Earth's surface, even through clouds and at night.

Data Application of the Month: River Flooding | UN-SPIDER Knowledge Portal

2. Change Detective — Detect What Changed

Compare radar images from two or more dates to identify changes such as flooding, water expansion, and land-surface movement. Highlight detected changes on an interactive map and calculate affected areas where the data supports it.

NHESS - Assessing riverbank erosion in Bangladesh using time series of Sentinel-1 radar imagery in the Google Earth Engine

3. Earth's Timeline — Understand the Changes

Explore a time slider or animation to see how an area changes over time. Investigate possible causes using supporting information such as rainfall, elevation, and river data. Learn how these changes affect land, rivers, agriculture, and communities.

বাংলাদেশের বর্তমান বন্যা পরিস্থিতি এবং আমাদের করনীয় | Protidiner Chitro BD | Latest Online Bangla News | প্রতিদিনের চিত্র বিডি | অনলাইন বাংলা সংবাদপত্র

4. Impact Missions — Turn Findings into Action

Complete short missions based on your radar discoveries. Identify affected roads, buildings, or settlements, answer investigation questions, and explore simple simulated solutions for safer communities. Earn XP, coins, badges, and research journal entries.

Example Mission: The Changing Jamuna River

Coherent, Not Chaotic, Migration in the Brahmaputra-Jamuna River - Eos
Jamuna River: Flood and permanent water in Bangladesh | Palash Basak, PhD posted on the topic | LinkedIn
NHESS - Assessing riverbank erosion in Bangladesh using time series of Sentinel-1 radar imagery in the Google Earth Engine
8

Imagine receiving an urgent mission from Mission Control:

Mission: Investigate the Changing Jamuna

1. Explore radar observations of the Jamuna River from different dates.
2. Compare pre-flood and post-flood images.
3. Identify areas where water extent has changed.
4. Calculate the estimated changed or inundated area.
5. Investigate possible impacts on nearby settlements and farmland.
6. Submit your findings and earn rewards.

The mission should use actual NISAR observations where suitable scenes are available. Sentinel-1 can be used for supplementary demonstrations, but its results must be clearly identified as Sentinel-1 rather than NISAR.

How the game works

EXPLORE

Choose a location and view radar images

DETECT

Compare images and identify changes

UNDERSTAND

Investigate causes and environmental impacts

COMPLETE

Submit findings, earn rewards, unlock missions

What makes TERRALENS interesting?

- Real satellite data: Missions are connected to actual Earth observations.
- Interactive investigation: Players compare radar images instead of simply reading information.
- Visual discoveries: Change maps, swipe comparisons, and time-based animations make surface changes easier to understand.
- Gamification: XP, coins, badges, mission levels, and a personal research journal encourage continued exploration.

Why this fits the Dancing with the SARs challenge

Challenge requirement	TERRALENS feature
Explore NISAR SAR data	Earth Explorer
Detect surface changes	Change Detective
Analyze changes over time	Earth's Timeline
Explain physical and human impacts	Impact Missions

My recommendation: Keep these four features as the complete core of TERRALENS. Avoid making a large village-building game, complex rescue simulation, or advanced flood simulator at this stage. Those could distract from the challenge's main objective: showing how NISAR radar observations reveal changes on Earth.

The strongest demonstration would be a before-and-after radar comparison of a real location, an interactive change map, a time-based view, and an automatically calculated area of change. That gives the project a clear scientific purpose while keeping the gameplay engaging and feasible.

TERRALENS — Complete Game Development Roadmap

A beginner-friendly strategy for building a playable, science-based adventure game using NASA–ISRO NISAR data.

My recommendation is to develop TERRALENS as a 2D satellite investigation adventure game with an interactive scientific map, rather than a traditional 3D open-world game.

Think of it as a combination of:

- Pokémon GO: Exploration, discoveries, missions and rewards.
- Subnautica: Exploring an unfamiliar environment and uncovering mysteries.
- Monument Valley: Beautiful, simple visual design and guided progression.
- GeoGuessr: Location-based investigation and observation.
- A scientific detective game: Collecting evidence, comparing observations and solving environmental mysteries.

You don't need to build an enormous game. You need to make the scientific investigation itself feel like an adventure.

Part 1. Understand what you are building

Before writing code, understand the complete gameplay loop.

The TERRALENS gameplay loop

MISSION CONTROL

An environmental mystery is detected.

EXPLORE

Select a location and examine satellite images.

INVESTIGATE

Compare dates and identify surface changes.

UNDERSTAND

Explore causes, affected areas and evidence.

COMPLETE THE MISSION

Submit your findings, earn XP and unlock new missions.

The most important development principle: Real satellite data should determine what the player investigates. The game mechanics, animations, characters and rewards should make that scientific investigation enjoyable.

Recommended technology stack

Don't start by learning ten different programming languages. Use one primary language for your game and a small number of tools for data processing and visualization.

Development area	Recommended tool	Purpose
Game development	Godot 4	Game screens, missions, characters and gameplay
Game scripting	GDScript	Game logic and interactions
Scientific processing	Python	Processing SAR images and detecting changes
Geospatial processing	Rasterio, GDAL, NumPy	Working with satellite data
Machine learning	scikit-learn, PyTorch	Optional classification models
Interactive maps	MapLibre GL JS	Satellite investigation maps
Backend	FastAPI	Connect the game to scientific processing
Database	PostgreSQL + PostGIS	Save missions, locations and results

Image storage	Cloudflare R2 or an institutional object store	Store processed satellite images
UI and illustration	Figma, Inkscape, Krita	Design the game interface and artwork
Version control	GitHub	Store and manage your code

Most of these tools are open-source or offer free tiers. Some hosting and storage services have usage limits or paid plans.

For a beginner, my preferred combination is:

Recommended stack

Godot + Python + FastAPI + PostgreSQL/PostGIS + MapLibre.

You can build the first version without machine learning, expensive cloud infrastructure or 3D graphics.

Part 2. Game design — how to make TERRALENS interesting

Step 1. Design the game world

Create a fictional research organization called TERRALENS Research Division.

The player is a newly recruited Earth Investigator. They receive satellite investigation assignments from Mission Control.

The game world should have five main visual elements:

Navigating the Future: OneWebs Game-Changing Astra Positioning and Timing Service

Mission Control

Mission selection and story progression.
Geography & Travel Bidibidi 2026

Earth Map

Discover locations and unlock investigations.
Satellite Imagery Gallery | Grain Data Solutions Inc.

Radar Laboratory

Examine actual satellite observations.
Cyber Detective | Devpost

Research Journal

Store evidence and completed discoveries.

You can use a simple 2D interface with these five areas. There is no need to construct a 3D world.

Step 2. Design the four core features

Each feature should contain a few gameplay mechanics, not just informational pages.

Feature 1 — Earth Explorer

| Viết bởi

Purpose: Let players discover investigation locations.

Players receive access to a world map with unlocked and locked investigation locations.

What to include:

1. Interactive world map with investigation markers.
2. Location information: name, event, observation dates.
3. Radar image viewer.
4. Before-and-after date selection.
5. Short introductory mission story.

Interesting gameplay mechanics:

- Unknown locations appear as mystery markers.
- Players unlock new locations by completing previous missions.
- Each location contains a hidden environmental mystery.
- A short transmission from Mission Control introduces each mission.

Example:

"Investigator, an unusual signal has been detected near the Jamuna River. Water coverage appears to have changed. Your assignment is to investigate the area."

Feature 2 — Change Detective

Satellites Revealed the Extent of the Flooding in the Pamarys Region

Purpose: Make radar investigation the actual gameplay.

This should be the most interactive and important screen in TERRALENS.

What to include:

1. Before-and-after radar image comparison.
2. Swipe slider to compare two images.
3. Toggle between before, after and change-detection layers.
4. Interactive map where players mark suspected changed areas.
5. Automatically calculated area statistics where the data supports them.

Interesting gameplay mechanics:

- Find the hidden floodwater.
- Identify areas where water expanded.
- Locate possible riverbank changes.
- Mark the suspicious region.
- Collect evidence points for correct observations.

For example, the player sees two radar images and receives this task:

"Three regions have experienced changes. Identify at least two regions where new water coverage is detected."

The game can highlight the player's selected regions and compare them with the scientifically generated reference result.

Feature 3 — Earth's Timeline

我院硕士生在地学Top期刊《International Journal of Applied Earth Observation and Geoinformation》发表洪水监测论文

Purpose: Show how Earth's surface changes over time.

This feature transforms a collection of satellite images into a story.

What to include:

1. Timeline slider.
2. Radar observations from multiple dates.
3. Animated transition between observations.
4. Change statistics over time.
5. Supporting information such as rainfall or river levels, when available.

Interesting gameplay mechanics:

- Identify when flooding first appeared.
- Find the date when water coverage was highest.
- Determine whether water coverage increased or decreased.
- Investigate whether the change was temporary or persistent.

For example:

"At which observation date did the floodwater cover the largest area?"

Players move the timeline and select the correct date.

Feature 4 — Impact Missions

ESA - BlackShore Cerberus map

Purpose: Connect scientific discoveries to real-world consequences.

Players use their findings to answer environmental investigation challenges.

What to include:

1. Nearby settlements, roads and agricultural land.
2. Affected-area identification.
3. Investigation questions.
4. Simple simulated decisions.
5. Mission report submission.

Interesting gameplay mechanics:

- Identify settlements located near a detected flood region.
- Find potentially affected farmland.
- Select the most exposed road.
- Recommend which area needs further inspection.
- Submit a short scientific conclusion.

Important: A radar image alone does not prove that a specific building or road is damaged. Display these as potential exposure or areas requiring investigation unless independently verified data supports a stronger claim.

Step 3. Create the 3 Game Levels

Instead of creating ten levels, TERRALENS will have three progressive levels. Each level teaches a new skill and helps players become better satellite investigators.

Level 1 — Radar Rookie

Beginner

- Story: Welcome to TERRALENS! Mission Control needs a new radar investigator.
- Task: Learn the basics of satellite radar images.
- Challenge: Identify rivers, water and land by recognizing bright and dark areas in a radar image.
- Reward: 100 XP + Radar Rookie badge.

Level 2 — Change Detective

Intermediate

- Story: A river region may have experienced a flood.
- Task: Compare radar images captured before and after the event.
- Challenge: Identify newly flooded areas, mark the changes and estimate the affected area.
- Reward: 250 XP + Change Detective badge.

Level 3 — Master Investigator

Advanced

- Story: Mission Control has discovered major changes in an area and needs a complete investigation.
- Task: Analyze radar observations from multiple dates.
- Challenge: Track environmental changes, identify potentially affected farmland or settlements, and submit an investigation report.
- Reward: 500 XP + Earth's Guardian badge + Research Certificate.

Step 4. Use the Same Structure for Every Mission

To make development easier, all three levels will follow the same six steps.

Mission stage	What happens
1. Briefing	Introduce the mission and objective.
2. Explore	Open the satellite location and radar images.
3. Investigate	Compare radar observations.
4. Collect Evidence	Mark important changes.

5. Answer	Answer a few investigation questions.
6. Submit	Complete the mission and earn rewards.

Three simple gameplay mechanics:

1. Hints: Give players 2–3 hints per mission.
2. Evidence Stars: Reward players for correctly identifying changes and dates.
3. Increasing Difficulty: Make each level slightly more challenging.

Avoid countdown timers. Reward careful investigation and correct observations instead of speed.

Step 5. Player Ranking and Reward System

Use XP to increase player ranks, coins for cosmetic items, and badges to celebrate achievements.

Player ranks

Rank	XP required
Radar Rookie	0–99
Signal Explorer	100–349
Change Detective	350–599
Earth Investigator	600–849
Earth's Guardian	850+

How players earn XP

Player action	XP
Complete a mission	100
Identify the correct changed region	50
Identify the correct observation date	30
Estimate the changed area correctly	50
Complete without hints	25

Players can also earn coins for completing missions. Coins can unlock:

- Investigator avatars.
- Radar laboratory themes.
- Map markers and profile decorations.

Coins should only unlock cosmetic items, not scientific answers or advantages.

Research Journal

Every completed mission automatically creates a journal entry containing:

- Mission name and location.
- Satellite and observation dates.
- Before-and-after radar images.
- Detected change map.
- Estimated changed area.
- Player's conclusion.

This journal becomes the player's personal record of completed Earth investigations.

Part 3. Research and scientific backend

This is the most technically important part of TERRALENS.

Before developing the game, you need to understand how satellite data becomes something a player can interact with.

Step 6. Understand the satellite data you need

You mentioned three useful data sources. I recommend using them in this order.

NISAR in Orbit (Artist's Concept) - NASA Science

1. NISAR

Primary mission data

The actual satellite data your game is built around.

NISAR's L-band radar is especially useful for studying changes in land, vegetation, wetlands and surface movement. The mission also carries an S-band radar.

Official resources: NASA NISAR · NISAR data products

ESA - Artist’s impression of Sentinel-1A

2. Sentinel-1

Development and training data

Use it to develop and test the processing pipeline before relying on NISAR.

Sentinel-1 provides C-band radar observations and a long archive. It is excellent for developing flood detection and change-detection workflows.

Access: ASF SAR search documentation.

Blue Marble: Next Generation with Topography and Bathymetry - NASA Science

3. NASA GIBS

Supporting visual data

Use it to provide geographical and optical context.

GIBS imagery can help players understand the surrounding landscape. It is not a substitute for SAR analysis.

Important NISAR data update

As of November 2026, the situation is better than the earlier assumption that NISAR data would only become available at the end of 2026.

- Public NISAR L-band provisional data began releasing on July 20, 2026.
- The initial provisional collection covers acquisitions from June 17, 2026 onward.
- These products are calibrated and partially validated, but still have known limitations.
- The archive is being expanded and reprocessed. Check the actual scene availability for your selected location and dates before planning a mission.
- NISAR S-band products are also being distributed through ISRO's Bhoonidhi portal.

Sources:

Alaska Satellite Facility+3

My advice: Build your initial working model using Sentinel-1, but make the processing pipeline sensor-aware so that NISAR can be substituted or added without rebuilding the game.

Step 7. Research the actual environmental event

Don't begin by downloading random satellite images.

Choose one event and one geographical area for your first playable mission.

I recommend:

Coherent, Not Chaotic, Migration in the Brahmaputra-Jamuna River - Eos
eos.org

First case study: Jamuna River flood and water expansion

- Study area: Jamuna River, Bangladesh.
- Main event: Flood-related water expansion.
- Main dataset: Sentinel-1 GRD.
- Target dataset: NISAR L-band, if suitable acquisitions are available.
- Supporting data: Rainfall, elevation, river boundaries and settlement layers.
- Main outcome: Before-and-after comparison and estimated new-water area.

Your research should answer these questions:

1. Which dates represent the pre-event and post-event conditions?
2. Are both satellite observations suitable for comparison?
3. What radar polarization is available?
4. How do water and land appear in the radar data?
5. What threshold or classification method can distinguish water?
6. How accurate is the estimated changed area?
7. What independent information can verify the results?

For Bangladesh, you can investigate rainfall information from NASA GPM and gauge records from the Bangladesh Water Development Board. Availability, access permissions and temporal resolution must be checked for the selected event.

Step 8. Collect the datasets

Here is the recommended research toolkit.

Dataset/tool	Why you need it	Access
ASF DAAC	NISAR and Sentinel-1 data discovery	Free
ASF Vertex	Search and visually inspect scenes	Free
Earthdata Login	Authenticate and download NASA data	Free
asf_search	Search and download data using Python	Free
Google Earth Engine	Rapid geospatial experimentation	Free for eligible noncommercial research, subject to terms
NASA GPM	Rainfall observations	Free
Copernicus DEM	Elevation information	Free
OpenStreetMap	Roads and settlement context	Free under ODbL
BWDB	River gauge observations	Check access and reuse conditions

Useful official resources:

- NISAR data access guide

- ASF data discovery
- ASF Search API
- ISRO NISAR S-band data release

Can you train your initial model using Sentinel-1?

Yes — recommended

But there is one important distinction.

You can train a model on Sentinel-1 and use it to develop a workflow for NISAR. You should not assume that a model trained on Sentinel-1 will produce accurate results on NISAR without adaptation and validation.

Why?

Sentinel-1	NISAR
C-band radar	L-band and S-band radar
Different wavelength	Different wavelength
Different vegetation response	Different vegetation response
Different backscatter characteristics	Different backscatter characteristics

Your first model should therefore be developed in stages:

1. Train or develop the water classification workflow with Sentinel-1.
2. Test it on separate Sentinel-1 scenes.
3. Obtain NISAR scenes covering the same or a comparable environment.
4. Calibrate or adapt the method for NISAR.
5. Validate the NISAR result using independent reference information.
6. Display the sensor, processing method and data quality in the game.

You can also use a simple threshold-based method first. Machine learning is not compulsory.

Step 9. Build the SAR processing pipeline

This is where your satellite data becomes an actual game feature.

I recommend using Python for all satellite processing. The game should not process large satellite files itself.

9.1 Install the scientific environment

Install Python and create a separate development environment.

Recommended libraries:

```
None
pip install numpy scipy pandas
pip install rasterio geopandas shapely
pip install matplotlib scikit-image
pip install asf_search
pip install scikit-learn
pip install fastapi uvicorn
pip install rio-tiler
```

Also install:

- QGIS — desktop GIS visualization.
- SNAP — ESA's Sentinel Application Platform for SAR processing.
- GDAL — geospatial data processing.

All are free and open-source.

9.2 Download the satellite images

Use ASF's Python library to search for available scenes.

Your workflow:

1. Define the study area using a geographical polygon.
2. Search for Sentinel-1 GRD scenes.
3. Filter by acquisition dates.
4. Select suitable polarization and orbit direction.
5. Download pre-event and post-event scenes.
6. Repeat the discovery process for NISAR.
7. Store the original files separately from processed files.

For NISAR, inspect the product type and maturity level before selecting data. For example, geocoded products such as GSLC or GCOV can be useful for visualization and analysis, depending on the task.

You can start with the ASF Python documentation: asf_search guide.

9.3 Process the radar images

The processing steps depend on the input product.

For Sentinel-1 GRD, your workflow should generally include:

1. Apply or verify radiometric calibration.
2. Remove thermal noise where required.
3. Apply suitable speckle filtering.
4. Apply terrain correction and geocoding.
5. Convert values into a consistent representation, such as decibels where appropriate.
6. Align the two dates to the same grid.
7. Clip both images to the study area.

For NISAR, use the appropriate mission product and its metadata. Do not blindly apply the Sentinel-1 GRD preprocessing sequence to NISAR products that are already calibrated or geocoded.

Important: Make sure the images are aligned to the same coordinate reference system, pixel size and extent before comparing them.

9.4 Develop your first flood detection algorithm

Do not start with deep learning. Start with threshold-based classification.

Your supplied code is a good prototype:

```
None
import numpy as npfrom scipy import ndimagedef water_mask(db_array, threshold_db=-16.0, min_pixels=50):    mask = db_array < threshold_db    labels, count = ndimage.label(mask)    sizes = ndimage.sum(        mask,        labels,        range(1, count + 1)    )    keep = np.isin(        labels,        np.flatnonzero(sizes >= min_pixels) + 1    )    return keepdef inundated_km2(    before_mask,    after_mask,    pixel_m=10.0):    new_water = after_mask & ~before_mask    area = (        new_water.sum()        * pixel_m**2        / 1_000_000    )    return float(area)
```

Understand what the algorithm does:

- `water_mask()` identifies pixels that may represent water.
- `threshold_db` controls the backscatter threshold.
- `min_pixels` removes small isolated groups.
- `inundated_km2()` estimates the area classified as new water.

The formula is:

$$A = N \times \frac{p^2}{10^6}$$

Where:

- A = estimated area in km².
- N = number of newly classified water pixels.
- p = pixel size in metres.

However, there are three scientific improvements you should make before using this in the final game:

Improvement 1 — Don't use one universal threshold.

A threshold of -16 dB is a starting experiment, not a universal water-detection rule. Water backscatter varies with sensor, polarization, wind, surface conditions, vegetation and terrain.

Improvement 2 — Handle permanent water.

A pixel that was already water before the flood should not be counted as newly inundated. Use a permanent-water reference mask or a suitable pre-event baseline.

Improvement 3 — Calculate area using actual pixel dimensions.

The `pixel_m` formula assumes square pixels with a known ground resolution. For projected rasters, calculate pixel area using the raster transform. For geographic rasters, use an appropriate geodesic or equal-area calculation.

Also consider:

- Terrain shadow and radar layover.
- Vegetation-covered floodwater.
- Wet soil that may be mistaken for water.
- Image registration errors.
- Missing data and quality masks.

9.5 Add a machine learning model later

Once your threshold-based system works, you can experiment with machine learning.

Here are the models I recommend:

Model	Use in TERRALENS	Difficulty
Otsu thresholding	Automatic initial segmentation	Easy

Random Forest	Water vs land classification	Easy–medium
Support Vector Machine	Classification using radar features	Medium
U-Net	Pixel-level flood segmentation	Advanced
Change Vector Analysis	Comparing radar measurements	Medium
Change detection using coherence	Detecting changes in surface scattering properties	Advanced

My recommendation:

Start here
Random Forest or threshold-based classification.

Use features such as VV, VH, backscatter differences, polarization ratios and suitable terrain information when available.

Later, you can train a U-Net using labelled flood masks if you have enough reliable training data.

For your first competition demonstration, a scientifically validated threshold-based method is more valuable than an impressive but unvalidated neural network.

9.6 A crucial issue: training and testing NISAR

Do not simply combine Sentinel-1 and NISAR images into one training dataset and assume they are interchangeable.

A better experimental design is:

- Train a baseline model using Sentinel-1.
- Evaluate it on held-out Sentinel-1 locations and dates.
- Develop a NISAR-specific baseline.
- Compare the two sensors separately.
- Investigate whether fine-tuning or feature normalization improves transfer.
- Evaluate NISAR results against independent reference labels.

This also gives you a strong research question:

How well can a flood detection workflow developed using C-band Sentinel-1 observations be adapted to L-band NISAR observations?

That can become an additional scientific contribution of TERRALENS.

Step 10. Turn the results into visualizations

Your game should not display raw scientific outputs without explanation.

It should turn them into interactive, understandable visual layers.

I recommend these five outputs:

The five investigation visualizations

Space Radar Image of Manaus Region of Brazil - NASA Science
science.nasa.gov

1. Original SAR image

Show the actual radar observation and its acquisition date.
Automatic Flood Detection from Satellite Images Using Deep Learning | by Ömer Buğrahan Çalışkan | Medium
medium.com

2. Before-and-after comparison

Let players move a swipe divider between two observations.
The Use of Sentinel-1 Time-Series Data to Improve Flood Monitoring in Arid Areas
mdpi.com

3. Change detection map

Highlight new water, persistent water and non-water regions.
Research | Dr. Zhiwei Li - Homepage
zhiweili.net

4. Time-series animation

Show how the classified area changes across observation dates.
UNOSAT
unosat.org

5. Impact and statistics dashboard

Display estimated area, potentially exposed features and data limitations.

Which visualization tools should you use?

Tool	What to use it for	Recommendation
QGIS	Inspect satellite images, validate results and create maps	Essential
Matplotlib	Generate charts and static scientific images	Essential
Folium	Quickly prototype interactive geographical maps	Optional
MapLibre GL JS	Interactive maps and raster overlays	Recommended
Leaflet	Lightweight interactive web maps	Alternative
Google Earth Engine	Rapid satellite processing and prototyping	Optional

MapLibre is particularly useful because it supports raster tiles, vector overlays and interactive geographic layers.

MapLibre GL JS+1

NASA GIBS is useful for optical and other NASA browse imagery through public WMTS and XYZ services.

Global Imagery Browse Services (GIBS)

My recommendation: Use QGIS to inspect the scientific results, Python to generate the results, and MapLibre to show them inside the game.

Step 11. Build your backend

Think of your backend as the scientific brain of TERRALENS.

The frontend asks for a mission. The backend finds the corresponding processed satellite images, runs or retrieves the analysis, and returns the results.

The recommended backend architecture

GODOT GAME

Player selects a mission and investigates

HTTP requests / JSON

FASTAPI BACKEND

Mission API · Player API · Analysis API

PostgreSQL

Player progress, missions, results

Raster storage

COGs, masks, maps, outputs

ASF / NISAR

Satellite data discovery

Python analysis

Detection and area calculations

MAPLIBRE VISUALIZATION

Radar images, change layers and statistics

Do you need external APIs?

Yes, but you don't need a large collection of APIs.

You need a small set of services:

API or service	What it does	Required?
ASF Search API / asf_search	Discover and retrieve SAR scenes	Yes, for your chosen ASF workflow
NASA Earthdata authentication	Access protected data downloads	Yes, where required

FastAPI endpoints	Connect the game to your backend	Yes
Map tile service	Serve processed radar layers	Yes for interactive raster maps
PostgreSQL/PostGIS	Store mission and geographic information	Recommended
Rainfall API/data	Provide environmental context	Optional
OSM data/services	Provide roads and settlements	Optional

You do not need to connect the frontend directly to ASF or NASA Earthdata. Put those connections behind your own backend.

That way, the frontend does not need to know how to download or process satellite files.

11.1 Create your backend API

Install FastAPI:

```
None
pip install fastapi uvicorn
```

Create a file named `main.py`:

```
None
from fastapi import FastAPIapp = FastAPI(title="TERRALENS
API")@app.get("/")def home():    return {"message": "Welcome to
TERRALENS"}@app.get("/missions")def get_missions():    return [
{            "id": 1,            "name": "Changing Jamuna",
"difficulty": "Beginner"        }
]@app.get("/missions/{mission_id}")def get_mission(mission_id:
int):    return {        "id": mission_id,        "location":
"Jamuna River, Bangladesh",        "status": "available"
```

```
}@app.get("/analysis/{mission_id}")def get_analysis(mission_id: int):    return {        "changed_area_km2": 12.5,
"sensor": "Sentinel-1",        "result": "New water detected"
}
```

Run it:

```
None
uvicorn main:app --reload
```

Open:

`http://127.0.0.1:8000/docs`

You will see an interactive API documentation page.

The `12.5` in this beginner example is a placeholder, not an actual Jamuna measurement. Your final endpoint must retrieve the result from the processing pipeline.

11.2 Design the API endpoints

Eventually, your backend can have these endpoints:

Endpoint	Purpose
`GET /missions`	Get all available missions
`GET /missions/{id}`	Get mission details
`GET /scenes/{id}`	Get satellite scene metadata
`GET /imagery/{id}`	Get image or tile information

`GET /analysis/{id}`	Retrieve change detection results
`POST /investigations/{id}/submit`	Submit player findings
`GET /players/{id}/progress`	Retrieve player progress
`GET /players/{id}/journal`	Retrieve research journal
`GET /players/{id}/rank`	Retrieve player rank

For long-running image processing, use background jobs instead of keeping an API request open while large files are being processed.

11.3 Organize your database

Use PostgreSQL with PostGIS.

Create these basic tables:

- `players`
- `missions`
- `satellite_scenes`
- `analysis_results`
- `player_submissions`
- `player_rewards`
- `research_journal`

For example, an analysis result should store:

```
None
{
  "mission_id": 1,
  "sensor": "Sentinel-1",
  "product_type": "GRD",
```

```
  "before_date": "2026-07-01",
  "after_date": "2026-07-13",
  "changed_area_km2": 12.5,
  "method": "Threshold classification",
  "threshold_db": -16.0,
  "status": "experimental"
}
```

Again, these dates and measurements are illustrative placeholders. Your database should contain the actual scene metadata and calculated results.

Also store:

- Scene IDs and product versions.
- Polarization and orbit direction.
- Processing parameters.
- Pixel resolution and CRS.
- Quality flags and validation metrics.
- Data provenance and attribution.

This is important because two investigators should be able to reproduce the same analysis.

11.4 Store satellite images efficiently

Don't put large GeoTIFF files inside your database.

Instead:

1. Store the original satellite data in a separate archive.
2. Convert suitable processed images to Cloud Optimized GeoTIFF (COG).
3. Store COGs in local storage during development.
4. Later use object storage.
5. Serve image tiles through a tile server such as TiTiler/rio-tiler.
6. Store only metadata and file references in PostgreSQL.

Your proposed processing structure is excellent:

```
None
ASF / NISAR / Sentinel-1
          |
          v
  Data discovery and download
          |
```

```
      v
SAR preprocessing
      |
      v
Processed COG files
      |
      +------------------+
      |                  |
      v                  v
Tile server         Python analysis
      |                  |
      v                  v
Radar imagery       Change maps
                         |
                         v
                  Area statistics
      |                  |
      +--------+---------+
               |
               v
         FastAPI backend
               |
               v
         TERRALENS game
```

This architecture lets you update satellite data without rewriting the game.

Part 4. Frontend — how to build the actual game

Now let's discuss what the player sees when opening TERRALENS.

Step 12. Design the complete screen structure

I recommend creating seven main screens.

Screen 1 — Welcome / Login

terrascopelabs.in

Include:

- TERRALENS logo.
- Animated Earth background.
- Start Investigation button.
- Login and guest access.
- Short introduction to the game.
  Purpose: Introduce the game and create excitement.

Screen 2 — Mission Control Dashboard

OrbitTrack — Real-Time Satellite Mission Control Dashboard by Scorpion on Dribbble
dribbble.com
Include:

- Player avatar and rank.
- XP progress bar.
- Coins and badges.
- Current mission.
- New mission notifications.
- Earth exploration button.
- Research journal shortcut.
  Purpose: Give players a central place to manage their journey.

Screen 3 — Earth Explorer

OrbitLab | TakeMe2Space | A portal to orbital compute & storage
tm2.space
Include:

- Interactive world map.
- Location markers.
- Locked and unlocked missions.
- Location search.
- Mission details when a marker is selected.
- Explore button.
  Purpose: Help players choose their next investigation.

Screen 4 — Radar Laboratory

SAR Change Detection
geospace-solutions.com
Include:

- Original radar image.
- Before-and-after comparison.
- Image date selector.
- Sensor and polarization information.
- Swipe comparison tool.
- Layer selector.
- Short radar interpretation guide.
  Purpose: Make the main scientific investigation interactive.

Screen 5 — Change Detection

Synspective Announces the Release of SAR-based Flood Damage Assessment

Solution Service - Synspective
synspective.com
Include:

- Detected change layer.
- Marking or drawing tools.
- New water and permanent water layers.
- Estimated changed area.
- Player answer panel.
- Submit findings button.

Purpose: Let players interact with and interpret scientific results.

Screen 6 — Mission Results

SCI-FI UI ASSET PACK [O.R.I.O.N.] | GameDev Market
mail.gamedevmarket.net
Include:

- Mission completion animation.
- Correct and incorrect findings.
- Scientific accuracy feedback.
- XP and coin rewards.
- Newly unlocked badges.
- Next mission button.

Purpose: Make each completed investigation feel rewarding.

Screen 7 — Research Journal / Player Profile

Save 30% on Quantum Odyssey on Steam
store.steampowered.com
Include:

- Player rank and statistics.
- Completed missions.
- Saved satellite images.
- Investigation reports.
- Earned badges.
- Total XP and coins.
- Research certificate.

Purpose: Give players a reason to return and continue investigating.

Step 13. Choose your frontend development technology

There is one important architectural decision here.

You have two possible development approaches.

Option A — Godot game

صناعة الألعاب - أكاديمية حسوب - Godot تعرف على محرر محرك الألعاب جودو

Godot

Language: GDScript

Best for: A standalone 2D adventure game with custom animations, characters, game logic and scenes.

Advantages:

- Free and open-source.
- Excellent 2D game development.
- Built-in animation tools.
- Scene and node-based architecture.
- HTTPRequest nodes can communicate with FastAPI.

Disadvantage:

MapLibre is a JavaScript mapping library, whereas Godot uses its own rendering environment. Integrating a full interactive MapLibre map inside a native Godot game requires additional work, such as a web view or a custom map integration.

Option B — Web-based adventure game

[Phaser - A fast, fun and free open source HTML5 game framework]

Phaser + React

Language: JavaScript or TypeScript

Best for: An interactive game with scientific maps, dashboards, animations and browser accessibility.

Advantages:

- Easier integration with MapLibre.
- Easy connection with FastAPI.
- Works directly in a browser.
- Can be deployed as a web application.
- Can later be packaged as a mobile application.

Disadvantages:

- You need to manage some game systems yourself.
- You need to learn JavaScript or TypeScript.

My final frontend recommendation

Best fit for TERRALENS

Use React + Phaser + MapLibre for the first competition version.

Why?

Your game is fundamentally a geospatial investigation application with game mechanics. Its main feature is not character movement or a complex 3D world. It is satellite image exploration.

React can manage the application screens, Phaser can manage adventure-style game interactions, and MapLibre can handle the scientific map.

This combination also makes it easier to build a browser-based demo that judges can access without installing anything.

You can still build a Godot version later if you want a more traditional game experience.

Step 14. How the frontend connects with the backend

You do not need to make the frontend understand satellite data processing.

It only needs to request and display the results.

For example:

Player action: Opens the Jamuna River mission.

Frontend sends:

```
None
GET /missions/1
```

Backend returns:

```
None
{
  "mission_name": "Changing Jamuna",
  "location": "Bangladesh",
  "sensor": "Sentinel-1",
  "before_image": "/tiles/before/{z}/{x}/{y}",
  "after_image": "/tiles/after/{z}/{x}/{y}",
  "change_layer": "/tiles/change/{z}/{x}/{y}"
}
```

Frontend displays:

- Before radar image.
- After radar image.
- Interactive change map.
- Mission questions.

- Estimated changed area.

When the player submits findings, the frontend sends the answers to FastAPI.

The backend evaluates them, updates the player's XP and saves the investigation in the journal.

This is the complete connection between gameplay and science.

Part 5. Illustrations, animations and game design

Step 15. Create the visual identity of TERRALENS

I recommend a futuristic satellite research theme.

Use:

- Dark navy background.
- Electric blue highlights.
- Cyan radar scanning effects.
- Green for detected land.
- Orange or red for unusual changes.
- White typography.
- Minimal glowing borders.

Your design should look like a combination of a satellite command center and an adventure game.

Tools for illustrations and animation

[Report: The Designer Class Deep Dive | Contrary Research]

Figma

Interface design

Design all seven screens, buttons, icons, layouts and prototypes. Free starter plan with limits.

[16 Best Free Graphic Design Software [2026]]

Krita

Illustration

Draw investigator characters, Earth backgrounds, radar-themed illustrations and mission artwork. Free and open-source.

[Inkscape vs Illustrator: Battle of the Vector Graphics Software | Envato Tuts+]

Inkscape

Vector graphics

Create scalable logos, icons, badges, location markers and interface elements. Free and open-source.

[[Extension] Link Duplicate Cels - Scripts & Extensions - Aseprite Community]

LibreSprite

2D game animation

Create animated characters, icons and sprite sheets. Free and open-source.

Blender

Optional 3D

Create satellite models, Earth animations, orbital effects and 3D introductory scenes. Free and open-source.

Step 16. Which animations should you actually create?

You don't need hundreds of animations. Make around eight reusable animations.

Animation	Where to use
Rotating Earth	Welcome screen
Satellite orbit	Mission Control
Radar scanning sweep	Radar Laboratory
Image transition	Before-and-after comparison

Change region glow	Change detection
Timeline transition	Earth's Timeline
XP reward animation	Mission results
Badge unlock animation	Player profile

For web development, use:

- CSS animations for buttons, glowing effects and transitions.
- Phaser animations for game elements.
- Lottie animations for lightweight interface effects.
- GSAP for more advanced timeline-based animations.

Don't animate the actual satellite pixels in a way that could misrepresent the data. Keep scientific image layers and decorative animations separate.

Part 6. How to develop everything in the correct order

This is where beginners often make a mistake: they start designing game screens before confirming that their satellite analysis actually works.

Don't do that.

Follow this development sequence.

Step 17. Divide development into eight phases

1. Phase 1 — Research and planning
   Week 1
   Tasks:
   - Finalize the Jamuna River case study.
   - Find suitable Sentinel-1 pre-event and post-event scenes.
   - Search for NISAR observations covering your selected area.
   - Study SAR backscatter and water detection.
   - Identify the mission's learning objectives.
   - Sketch the seven frontend screens.

2. Deliverable: Research plan, selected datasets and game wireframes.
3. Phase 2 — Satellite data processing
   Weeks 2–3
   Tasks:
   - Set up Python, QGIS and SNAP.
   - Download Sentinel-1 scenes.
   - Preprocess the satellite images.
   - Create before-and-after comparisons.
   - Implement threshold-based water detection.
   - Calculate the changed area.
   - Validate the results.
4. Deliverable: Two processed images, one change map and area statistics.
5. Phase 3 — Backend development
   Week 4
   Tasks:
   - Install FastAPI.
   - Create mission APIs.
   - Create analysis result APIs.
   - Set up PostgreSQL.
   - Store scene metadata and analysis results.
   - Create a basic player progress system.
6. Deliverable: Working backend that returns mission and analysis information.
7. Phase 4 — Interactive visualization
   Week 5
   Tasks:
   - Convert processed rasters to COGs.
   - Set up a raster tile server.
   - Connect MapLibre.
   - Add before-and-after comparison.
   - Add change detection layers.
   - Add the timeline.
8. Deliverable: A working interactive radar investigation screen.
9. Phase 5 — Game frontend
   Weeks 6–7
   Tasks:
   - Build the welcome screen.
   - Build Mission Control.
   - Build Earth Explorer.
   - Build Radar Laboratory.
   - Build Change Detective.
   - Build Mission Results.
   - Build Research Journal.
10. Deliverable: A playable game interface.
11. Phase 6 — Gameplay and rewards
    Week 8
    Tasks:
    - Add the first three levels.
    - Create investigation questions.

    - Implement answer checking.
    - Add XP and coins.
    - Add badges and player ranks.
    - Save mission progress.
12. Deliverable: A complete playable mission progression system.
13. Phase 7 — NISAR integration
    Weeks 9–10
    Tasks:
    - Obtain suitable NISAR products.
    - Inspect their product maturity and metadata.
    - Adapt the processing workflow.
    - Compare NISAR and Sentinel-1 results separately.
    - Validate the NISAR-based change detection.
    - Replace or supplement the demonstration imagery with NISAR results.
14. Deliverable: A scientifically documented NISAR investigation.
15. Phase 8 — Testing and final deployment
    Week 11
    Tasks:
    - Test every mission.
    - Check map loading performance.
    - Test the calculations.
    - Check player progression.
    - Add scientific data citations and limitations.
    - Deploy the application.
    - Prepare the competition demonstration.
16. Deliverable: A functioning TERRALENS prototype.

The schedule is a suggested 11-week plan for a small team. Data availability, programming experience and the number of developers may change the timeline.

Part 7. Recommended project folder structure

Create one main folder called `terralens`.

Organize it like this:

```
None
TERRALENS/
│
├── frontend/
│   ├── src/
│   │   ├── screens/
```

```
│   │   ├── components/
│   │   ├── game/
│   │   ├── maps/
│   │   ├── animations/
│   │   └── assets/
│   │
│   └── package.json
│
├── backend/
│   ├── main.py
│   ├── api/
│   ├── database/
│   ├── models/
│   ├── services/
│   └── requirements.txt
│
├── research/
│   ├── notebooks/
│   ├── preprocessing/
│   ├── detection/
│   ├── validation/
│   └── models/
│
├── data/
│   ├── raw/
│   ├── processed/
│   ├── cog/
│   └── results/
│
├── missions/
│   ├── level_01.json
│   ├── level_02.json
│   ├── level_03.json
│   └── ...
│
└── documentation/
    ├── research.md
    ├── api.md
    └── game_design.md
```

This separation will save you a lot of trouble as the project becomes larger.

Part 8. Free tools and services — your complete toolkit

Here is the toolkit I would actually use, avoiding unnecessary paid subscriptions.

Purpose	Tool	Cost
Coding	Visual Studio Code	Free
Version control	GitHub	Free tier
Game design	Phaser	Free, open-source
UI design	Figma	Free starter tier
Illustration	Krita	Free
Vector graphics	Inkscape	Free
Animation	Blender	Free
Satellite data	ASF DAAC	Free
Data processing	Python	Free
SAR processing	ESA SNAP	Free

GIS	QGIS	Free
Mapping	MapLibre	Free, open-source
Raster serving	rio-tiler / TiTiler	Free, open-source
Backend	FastAPI	Free, open-source
Database	PostgreSQL + PostGIS	Free, open-source
ML	scikit-learn / PyTorch	Free, open-source
Testing APIs	Bruno or Postman	Free tiers
Hosting	GitHub Pages / Cloudflare Pages	Free tiers
Backend hosting	Self-hosted or a suitable free-tier provider	Depends on limits
Database hosting	Local PostgreSQL or a suitable free-tier provider	Depends on limits

A word of caution: Free software does not automatically mean free deployment. Raster storage, bandwidth, compute and database hosting can become expensive if you serve large images to many players.

For the first version, process your scenes once and serve precomputed results. Don't run a deep-learning model every time a player opens a mission.

Part 9. How to make the first playable version without getting overwhelmed

If I were guiding you through your first development attempt, I would NOT ask you to build all ten levels.

I would ask you to build one complete mission first.

TERRALENS — First playable prototype

"I wish the river would vanish" — Shorthand Social
social.shorthand.com

Mission 01: The Changing Jamuna

Your minimum viable game

Screen 1 — Mission briefing

A river has experienced unusual changes. Mission Control asks the player to investigate.

Screen 2 — Radar exploration

Display one pre-event and one post-event radar image.

Screen 3 — Change detection

Player moves the comparison slider and identifies changed regions.

Screen 4 — Investigation challenge

Ask three questions:

1. Where did new water appear?
2. Which date shows the greatest water coverage?
3. What is the estimated area of newly detected water?

Screen 5 — Impact investigation

Display nearby settlements and farmland. Ask the player to identify potentially exposed areas.

Screen 6 — Mission results

Show:

- Correct findings.

- Estimated changed area.
- XP earned.
- Badge unlocked.
- Scientific explanation.

Screen 7 — Research journal

Save the mission and its findings.

What this one mission proves

- Real SAR data is integrated.
- Radar images are interactive.
- Changes are scientifically detected.
- Area calculations are performed.
- Players investigate and answer questions.
- Game rewards are connected to scientific achievements.

Once this works, duplicate the mission structure and change the data, location, story and questions for the remaining levels.

Part 10. Important things that can make or break TERRALENS

There are a few technical mistakes I want you to avoid.

Common mistake	What you should do instead
Building the whole game before testing satellite data	Prove one complete satellite analysis first
Using only sample radar pictures	Use real scenes and show their metadata
Treating Sentinel-1 and NISAR as identical	Preserve sensor-specific processing and validation
Using a fixed threshold everywhere	Calibrate and validate thresholds for each dataset

Showing only static images	Add swipe comparisons, map layers and timelines
Making every mission a quiz	Let players manipulate real satellite observations
Starting with a complex 3D game	Start with a 2D investigation interface
Downloading huge satellite files repeatedly	Preprocess once and cache COGs
Giving XP just for clicking buttons	Reward correct, evidence-based investigation
Claiming actual community damage from radar alone	Show potential exposure unless independently verified

Three additional features that would strengthen your scientific credibility

These don't need to be separate game features. They can be small additions to your existing four features.

1. Evidence panel

Every result should show the satellite name, sensor, date, polarization and processing method.

2. Confidence indicator

Display whether the result is experimental, validated or uncertain. Avoid presenting uncertain classifications as confirmed facts.

3. Scientific explanation

After each answer, explain why the result is correct.

For example:

"New water coverage was identified because the post-event observation contains pixels classified as water that were not classified as water in the pre-event observation."

This turns every mission into a learning experience.

Part 11. My final recommendation — what your team should do first

Don't distribute tasks randomly. Divide your team according to the technical components.

Team role	Main responsibility	Main tools
SAR researcher	Download, preprocess and analyze satellite data	Python, SNAP, QGIS
ML researcher	Develop and validate change detection	Python, scikit-learn
Backend developer	Build APIs, database and image-serving pipeline	FastAPI, PostgreSQL
Frontend developer	Build screens, interactive maps and game logic	React, Phaser, MapLibre
UI/UX designer	Create illustrations, layouts and animations	Figma, Krita, Inkscape

If your team is small, combine the research and ML roles, and combine backend and data-serving responsibilities.

The exact order of your first 10 development tasks

Your development checklist

0/10 completed

1. Install Python, VS Code, QGIS and SNAP

Prepare your scientific development environment.

2. Select the Jamuna River study area

Define your first mission's geographical boundary.

3. Search ASF for suitable Sentinel-1 scenes

Find actual pre-event and post-event observations.

4. Find suitable NISAR scenes

Check coverage, product type and maturity.

5. Process two Sentinel-1 images

Generate aligned and geocoded observations.

6. Create a flood change-detection algorithm

Generate a water mask and calculate changed area.

7. Validate the scientific output

Check the result against independent information.

8. Create the first interactive map

Display the images and change layer using MapLibre.

9. Build the first mission in the frontend

Create briefing, investigation and result screens.

10. Connect FastAPI and implement XP rewards

Save findings and make the mission playable.

Reset checklist

Useful official learning resources

These are the resources I would keep open while developing:

1. NISAR Data User Guide — Understand NISAR products.
2. ASF Python Search Guide — Find and download satellite observations.
3. Sentinel-1 GRD Dataset Guide — Learn about the initial development dataset.
4. NASA GIBS API Documentation — Display NASA imagery.
5. MapLibre Documentation — Create interactive maps.
6. FastAPI Tutorial — Learn backend development.
7. Phaser Documentation — Build the adventure game.

8. Godot Documentation — Alternative game development approach.
9. QGIS Documentation — Learn geospatial visualization.

My final advice

TERRALENS should be developed around one central achievement:

A player can open a real satellite observation, investigate changes, understand what happened, and submit a scientifically supported conclusion.

Everything else — levels, XP, coins, characters, badges, animations and journals — should make that experience more enjoyable.

If you have limited development time, prioritize these five deliverables:

1. One working Sentinel-1 before-and-after radar investigation.
2. One validated change-detection algorithm with area calculation.
3. One interactive map with comparison and timeline functionality.
4. One complete playable mission with XP and rewards.
5. One successful integration of actual NISAR data, clearly labelled by sensor and product maturity.

That would be a much stronger competition prototype than ten beautiful levels that only display simulated satellite images.

The key is to make NISAR the scientific engine of the game, not just an image displayed inside it.